

Solar Power Forecasting

Adarsh Chauhan

230107008



Final Project submission

Course Name: Applications of AI and ML for Chemical Engineering

Course Code: CL653

Submission Date: April 23, 2026

Contents

1	Executive Summary	3
2	Introduction	3
3	Methodology.....	4
4	Implementation Plan	9
5	Testing and Deployment	11
6	Results and Discussion.....	13
7	Conclusion and Future Work.....	17
8	References	19
9	Appendices	20
10	Auxiliaries.....	22

1 Executive Summary

This project develops a comprehensive Solar Power Generation Forecasting System that combines classical Machine Learning, Seasonal Time-Series modelling (SARIMA), and Deep Learning (LSTM) to accurately predict solar AC power output. Built on real operational data from two solar plants in India (Plant 1 and Plant 2), the system addresses one of the most pressing challenges in renewable energy infrastructure: the inherent unpredictability of solar generation due to weather variability.

The dataset spans 34 days of 15-minute interval measurements from two plants with 22 inverters each. The pipeline engineers 14+ features with explicit leakage avoidance — specifically, EFFICIENCY (which uses the target variable) and DAILY_YIELD (a cumulative within-day counter) are excluded. Five model classes are trained: Linear Regression, Decision Tree, Random Forest with GridSearchCV, SARIMA, and LSTM.

SARIMA is applied at two granularities: hourly plant-level aggregates (~816 data points, seasonal period $s=24$) as the primary model enabling full four-panel residual diagnostics, and daily aggregates (34 points, $s=7$) as an alternative with two-panel simplified residual analysis. The Random Forest achieves $R^2 > 96\%$ on held-out 15-minute data. A Streamlit web application, deployable in two commands with seven Python packages, offers an interactive hourly/daily SARIMA toggle accessible to engineers without ML expertise.

2 Introduction

Background: Solar power forecasting was selected for this project for three intersecting reasons: scientific richness, real-world urgency, and direct relevance to Chemical Engineering.

Scientifically, the problem sits at the intersection of time-series modelling, environmental physics, and machine learning. It requires understanding both the mathematical structure of sequential data and the domain physics — how irradiance, angle of incidence, module temperature, and atmospheric scattering interact to drive inverter output. No single model class dominates; the problem genuinely benefits from the full spectrum from classical statistics (SARIMA) to ensemble ML (Random Forest) to deep learning (LSTM), making it ideal for demonstrating AI/ML breadth in a single coherent project.

From an urgency standpoint, India is among the world's fastest-growing solar markets, targeting 500 GW of renewable capacity by 2030. The dataset used originates from a real Indian operational plant, making findings directly applicable. Forecasting errors translate to grid instability and financial penalties under India's Deviation Settlement Mechanism (DSM), where generators pay ₹3–12 per unit for deviations from declared schedules.

For this course specifically, Chemical Engineering and Process Industries are among the largest industrial electricity consumers. Accurate solar forecasting directly impacts batch scheduling, reactor temperature control, distillation reboiler management, and energy procurement strategy — all core Chemical Engineering concerns.

Problem Statement: Traditional solar forecasting methods — persistence models, climatological averages, and physics-only NWP simulations — share a fundamental limitation: they cannot model the complex, non-linear, temporally-dependent interactions between weather variables and actual inverter output. They also fail to adapt to plant-specific characteristics, inverter degradation, or short-term weather transients.

This project addresses four specific gaps:

- Short-term (15-minute to hourly) solar AC power prediction at both inverter level and plant level with quantified uncertainty
- Multi-model comparison: classical statistics (SARIMA) vs ensemble ML (Random Forest) vs deep learning (LSTM)
- Automated fault and anomaly detection from generation patterns, without additional sensor infrastructure
- Accessible deployment for non-ML-expert engineers through a web interface requiring no specialist software or SCADA integration

3 Methodology

Data Source:

Dataset: Kaggle — Solar Power Generation Data (public domain). URL: <https://www.kaggle.com/datasets/anikannal/solar-power-generation-data>

File	Contents	Resolution
Plant_X_Generation_Data.csv	DC_POWER, AC_POWER, DAILY_YIELD, TOTAL_YIELD per inverter (22 inverters per plant)	Every 15 minutes
Plant_X_Weather_Sensor_Data.csv	AMBIENT_TEMPERATURE, MODULE_TEMPERATURE, IRRADIATION (plant-level sensor)	Every 15 minutes

Two plants are supported via a PLANT_ID = 1 or 2 toggle. The generation file contains one row per (timestamp, inverter); the weather file contains one row per timestamp. Merge is performed on DATE_TIME after deduplicating the weather file.

Data Preprocessing:

- Datetime parsing with auto-format detection (dayfirst=True) for both CSV date formats
- Merge on DATE_TIME; weather CSV deduplicated before merge to prevent row multiplication
- Physical validity filter: rows where AC_POWER < 0 or DC_POWER < 0 are removed (sensor fault indicator)
- Null removal on five critical columns: AC_POWER, DC_POWER, IRRADIATION, AMBIENT_TEMPERATURE, MODULE_TEMPERATURE
- Deduplication on (DATE_TIME, SOURCE_KEY) pairs to remove any duplicate inverter records

Model Architecture:

ML Models — Linear Regression, Decision Tree, Random Forest

All three ML models are trained on 15-minute per-inverter data using an 80/20 temporal split (shuffle=False to preserve time order). The Random Forest uses GridSearchCV with TimeSeriesSplit(n_splits=5) — prevents future data appearing in training folds during hyperparameter search.

Model	Configuration	Key Characteristics
Linear Regression	StandardScaler + OLS; all leakage-free features	Interpretable baseline; assumes linear relationships; fastest to train
Decision Tree	max_depth=15, min_samples_split=10	Captures non-linearity; no scaling required; prone to overfit at high depth
Random Forest ★	n_estimators:[100,200]; max_depth:[10,20,None]; min_samples_split:[5,10]; max_features:[sqrt,log2]; TimeSeriesSplit(5)	Best tabular accuracy; feature importance; robust to outliers; recommended production model

SARIMA — Seasonal Time-Series Model

SARIMA (Seasonal AutoRegressive Integrated Moving Average) is the standard classical approach for time series with periodic structure. Solar generation has a strong daily seasonal pattern — the bell-curve shape peaking at solar noon — making SARIMA a principled choice.

SARIMA(p,d,q)(P,D,Q,s) notation:

- p=1, d=1, q=1: AR(1) + first-order differencing for stationarity + MA(1) for error correction
- P=1, D=1, Q=0: seasonal AR(1) + seasonal differencing to remove periodic mean; Q=0 (no seasonal MA needed)
- s: seasonal period — number of time steps in one full cycle (24 for hourly-daily; 7 for daily-weekly)

Config	Granularity	Points	Period s	Train pts	Residual Level	Analysis
SARIMA(1,1,1)(1,1,0,24) ★ Primary	Hourly plant totals	~816	24 (daily cycle)	~653	Full diagnostics (all valid)	4-panel
SARIMA(1,1,1)(0,0,0,0) ◦ Alternative	Daily plant totals	34	7 (weekly cycle)	27	2-panel analysis only	residual

Understanding Residual Analysis vs Full Diagnostics:

Both SARIMA configurations perform residual analysis — examining differences between model-fitted values and actual observations. The two levels differ in which statistical tests can be reliably applied at each sample size:

Analysis Component	Daily SARIMA (n=27)	Hourly SARIMA (n≈653)	Reason for difference
Residuals over time (bar plot)	Yes ✓	Yes ✓ (panel 1 of diagnostics)	Always valid; shows if errors have temporal patterns
Residual distribution (histogram)	Yes ✓	Yes ✓ (panel 2 of diagnostics)	Always valid; checks approximate normality
Normal Q-Q plot	No ✗ (unreliable)	Yes ✓ (panel 3 of diagnostics)	Requires 40+ points; tails are meaningless at n=27

ACF of residuals (correlogram)	No ✗ (unreliable)	Yes ✓ (panel 4 of diagnostics)	Confidence bands too wide at n=27; 50+ needed
Ljung-Box formal test	No ✗ (insufficient df)	Readable from ACF panel	Formal white-noise test needs sufficient degrees of freedom

The statsmodels plot_diagnostics() function always produces all four panels. For the daily SARIMA, panels 3 and 4 are computed but flagged as unreliable and not interpreted. For the hourly SARIMA, all four panels are valid and interpreted fully.

LSTM — Deep Learning Extension

Two-layer LSTM: LSTM(128, return_sequences=True) → Dropout(0.2) → BatchNorm → LSTM(64) → Dropout(0.2) → BatchNorm → Dense(32, ReLU) → Dense(1). Sequence length = 24 steps (~6 hours of 15-min data). Huber loss is used for robustness to zero-power nighttime outliers.

Original training used 50 epochs, which was identified as excessive: val_loss plateaus around 10–15 epochs, meaning the additional 35–40 epochs waste compute without improving accuracy. This was corrected to a maximum of 20 epochs with EarlyStopping(patience=5), reducing wall-clock training time approximately 3× while achieving equivalent or better accuracy through the best-weights restore mechanism.

Tools and Technologies:

Tool / Library	Version	Purpose
Python	3.10+	Core programming language
Pandas / NumPy	2.0 / 1.24	Data manipulation and numerical computing

Scikit-learn		1.3+	ML models, GridSearchCV, TimeSeriesSplit, metrics, permutation importance
Statsmodels		0.14+	SARIMA (SARIMAX), plot_diagnostics(), seasonal decomposition
TensorFlow Keras	/	2.13+	LSTM deep learning model (notebook only; not in Streamlit app)
Matplotlib Seaborn	/	3.7 / 0.12	EDA dashboards, model evaluation plots, day-wise profiles
Streamlit		1.32+	Web application with hourly/daily SARIMA toggle (7-package install total)
Joblib		1.3+	Model persistence — RF .pkl, LSTM scalers .pkl

4 Implementation Plan

Development Phases:

Phase	Task	Status
Phase 1	Project proposal, motivation analysis, problem definition, literature review, methodology design	Completed
Phase 2	Data pipeline: loading, merging, cleaning, feature engineering with leakage avoidance (EFFICIENCY and DAILY_YIELD excluded)	Completed

Phase 3	EDA: Spearman correlation, distributions, hourly AC profile, fault detection (CoV), weather trends, seasonal decomposition	Completed
Phase 4	ML models: LR, DT, RF+GridSearchCV + Gini importance + Permutation importance (10 repeats)	Completed
Phase 5	SARIMA hourly (primary, s=24, full diagnostics) + daily (alternative, s=7, 2-panel residual); LSTM extension (20 epochs)	Completed
Phase 6	Streamlit app: hourly/daily toggle, interactive EDA dashboard, comparison tab, JSON report download	Completed
Phase 7	Report: real-life applications, challenges, advantages vs existing systems, consistency review	Completed

Model Training:

- ML 80/20 temporal split (shuffle=False): last 20% of rows are the test set; no test rows seen during training or GridSearchCV hyperparameter search
- TimeSeriesSplit(n_splits=5): each fold uses only past data to validate future data; prevents temporal leakage during cross-validation
- SARIMA hourly: 80/20 temporal split (≈ 653 training hours, ≈ 163 test hours); forecast horizon = 2+ full seasonal cycles
- SARIMA daily: 27 training days, 7 test days; forecast horizon = 1 seasonal cycle
- LSTM: max 20 epochs, EarlyStopping(patience=5), ReduceLROnPlateau(factor=0.5, patience=4); MinMaxScaler applied to both features and target separately for correct inverse transformation

Model Evaluation:

Metric	Formula	Notes
MAE	$\text{mean}(\text{actual} - \text{predicted})$	Average absolute error in kW; intuitive physical unit; reported per-model

RMSE	$\sqrt{\text{mean}((\text{actual}-\text{predicted})^2)}$	Penalises large errors more than MAE; primary model selection criterion
R²	$1-\text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Fraction of variance explained; scale-independent; used as cross-model comparison indicator
MAPE (daytime only)	$\text{mean}(\text{actual}-\text{pred} /\text{actual}) \times 100$ where AC_POWER>0	Computed on daytime rows only; nighttime near-zero values excluded to avoid division artefact

5 Testing and Deployment

Testing Strategy:

Issue	Root Cause	Fix Applied
RF predict on X_train (bug)	Copy-paste error; training set used for evaluation; artificially inflated metrics	Explicit <code>rf.predict(X_test)</code> with comment; held-out test set only
EFFICIENCY feature leakage	$\text{EFFICIENCY} = \text{AC_POWER}/(\text{IRRAD} \times 1000)$ directly uses the target variable	Excluded from all ML and LSTM feature sets; reason documented in code and report
DAILY_YIELD feature leakage	Cumulative within-day counter encodes same-day future energy readings	Excluded; autoregressive lag features (t-1, t-3) provide safe historical signal

SARIMA memory on 15-min data	Fitting on 68,000 high-frequency rows causes memory overflow	Hourly aggregation: ~816 plant-level points; also improves seasonal signal quality
SARIMA diagnostics unreliable on daily data	27 training points; ACF and Q-Q confidence bands are too wide for inference	Hourly SARIMA (653 training pts) used as primary with full diagnostics; daily shows 2-panel residual analysis
ARIMA vs SARIMA inconsistency	Earlier versions used ARIMA in report but SARIMAX in code	SARIMA used consistently in all three artefacts; full audit completed
LSTM 50 epochs — excessive	val_loss plateaus at ~10–15 epochs; extra epochs waste compute with no benefit	Max 20 epochs + EarlyStopping(patience=5); best weights restored automatically
MAPE inflation by nighttime zeros	AC_POWER≈0 at night; division produces arbitrarily large values	MAPE computed on daytime rows only (AC_POWER>0); documented in all metric outputs

Deployment Strategy:

Option	Steps	Cost
Local (development)	pip install -r requirements.txt && streamlit run app.py (opens at http://localhost:8501)	Free
Streamlit Community Cloud	Push to GitHub → connect at share.streamlit.io → click Deploy (~2 minutes)	Free

Cloud VM (AWS/GCP/Azure)	Install requirements, open port 8501, run streamlit run app.py as background process	VM cost only (~₹500/month for smallest instance)
---------------------------------	--	--

Requirements: 10 packages (streamlit, pandas, numpy, matplotlib, seaborn, scikit-learn, statsmodels, joblib, jinja2, scipy). No GPU, no SCADA, no specialised hardware. Minimum Python 3.10. Install size ~120 MB. TensorFlow (for LSTM) is a notebook-only dependency; not required for the Streamlit app.

Ethical Considerations:

- Public Kaggle dataset; no personal, private, or commercially sensitive data processed
- System provides decision support only — no autonomous grid control, dispatch actions, or safety-critical decisions
- Model limitations (sample size, SARIMA diagnostic validity, scale mismatch) communicated clearly in the user interface through callout boxes
- 95% confidence intervals shown explicitly so grid operators are not misled by overconfident point forecasts
- Feature leakage decisions fully documented with root cause; all exclusions explained in code comments and report

6 Results and Discussion

Findings:

Observation	Implication for Modelling
Irradiation: highest Spearman ρ with AC_POWER (~0.90+)	Primary physical driver; top RF feature; motivates SARIMA seasonal structure (bell-curve peak)
Clear bell-curve hourly profile — peak at 10:00–15:00	Motivates SARIMA seasonal period $s=24$ (daily cycle); TOTAL_MINUTES_PASSED outperforms discrete HOUR for ML

Module temperature 5–20 °C above ambient throughout generation hours	DELTA_TEMPERATURE carries real thermodynamic signal; confirms heat transfer physics in data
CoV > 0.70 on 4+ days	Flags cloud-cover events and potential inverter fault days; actionable maintenance alerts
DC/AC ratio ≈93–95% during generation hours	Consistent with high-quality inverter efficiency specs; confirms data integrity and valid sensor readings
DC_POWER and AC_POWER Spearman $\rho > 0.99$	Near-perfect correlation; DC_POWER is a valid and strong predictor for AC_POWER without being the target itself

Comparative Analysis:

Model	MAE (kW)	RMSE (kW)	R² (%)	MAPE* (%)	Notes
Linear Regression	63.941	113.400	91.76	34.24	Interpretable baseline; linear assumption limiting
Decision Tree	21.461	62.024	97.54	6.21	Captures non-linearity; overfits without depth control

Random Forest (tuned)	13.104	40.532	98.95	3.95	Recommended production model; best ML accuracy
LSTM	36.441	76.946	96.21	21.60	Temporal; captures ramp patterns; slower to train

* MAPE on daytime rows only ($AC_POWER > 0$). Metric ranges are indicative across both plants; exact values depend on run and plant.

Reference Approach	Approach Used	This System's Advantage
Persistence + NWP (industry standard)	Physics-based atmospheric model	ML captures plant-specific non-linearities that NWP misses; no meteorological data service required
Single LSTM (common academic)	One LSTM on raw data, single plant	Multi-model comparison; feature engineering substantially improves accuracy; two plants; deployment-ready
Prophet (Facebook/Meta)	Additive trend + seasonal decomposition	SARIMA is more principled for short regular series; direct CI from posterior; no black-box components
Commercial SCADA fault detection	Fixed-threshold rule-based alerts	CoV-based detection is statistically adaptive to plant baseline; no threshold calibration needed

Single-model academic papers	Typically one model, one dataset, no deployment	Five models compared across two plants; reproducible pipeline; Streamlit web app delivered
-------------------------------------	---	--

Challenges and Limitations:

Limitation	Impact	Future Mitigation
34-day dataset only	SARIMA daily ($s=7$) has limited statistical power; annual/monsoon seasonality cannot be modelled	Multi-year dataset; auto_arima for order selection
No external weather forecast input	SARIMA is autoregressive only; cannot anticipate sudden weather changes before they appear in measurements	NWP integration as exogenous variable in SARIMAX
LSTM not in Streamlit app	Deep learning too slow for interactive web deployment without GPU	FastAPI backend with pre-trained LSTM; async predictions
MAPE on daytime only	Cannot directly compare with papers reporting all-hours MAPE	Document clearly; compute separately for comparison if needed

Single geographic location	Results may not generalise to plants in significantly different climates	Test on plants in different MNRE zones; transfer learning
CoV fault detection is heuristic	Fixed CoV thresholds may not adapt to seasonal baseline shifts	Replace with Isolation Forest or statistical process control charts

7 Conclusion and Future Work

This project delivers a production-ready solar power forecasting system that meets three simultaneous standards: mathematical rigour (TimeSeriesSplit cross-validation, explicit leakage avoidance for EFFICIENCY and DAILY_YIELD, uncertainty quantification via SARIMA 95% CI), engineering practicality (two-command deployment, interpretable feature importance, automated fault detection), and scientific honesty (model limitations clearly communicated, diagnostic validity tied explicitly to dataset size).

The choice of solar power forecasting at the intersection of renewable energy and Chemical Engineering reflects a genuine industrial need. Indian process industries are increasingly required to integrate renewable energy under net-zero commitments, but lack accessible, open-source forecasting tools that a plant engineer — not a data scientist — can operate and trust. This system closes that gap.

Key contributions: (1) leakage-free feature engineering framework with explicit exclusion of EFFICIENCY and DAILY_YIELD, both documented with root cause; (2) dual-granularity SARIMA with statistically valid full diagnostics on hourly data and simplified residual analysis on daily data; (3) CoV-based fault detection without additional instrumentation; (4) five-model comparison across two plants with consistent architecture and honest uncertainty quantification; (5) a Streamlit web application with hourly/daily SARIMA toggle deployable for free.

Suggested Future Directions

Enhancement	Expected Impact	Priority
auto_arima (pmdarima)	Automatically select optimal (p,d,q,s); removes manual order specification; tested across both plants	High
Temporal Fusion Transformer (TFT)	State-of-the-art multi-horizon forecasting with attention-based interpretability; outperforms LSTM on long sequences	High
RF + SARIMA ensemble (weighted)	Combine tabular (RF) and temporal (SARIMA) signals; expected 10–20% variance reduction	Medium
Multi-year dataset	Enable annual seasonality; LSTM learns summer/monsoon/winter patterns; SARIMA diagnostics more robust	High
FastAPI REST endpoint for LSTM	Pre-trained LSTM served via async API; enables SCADA/DCS integration for real-time inverter-level forecasting	High
Quantile regression	Probabilistic 15-min predictions at all time horizons; uncertainty bounds beyond SARIMA daily scale	Medium
Isolation Forest for fault detection	Replace CoV heuristic with unsupervised anomaly detection;	Medium

	adapts to seasonal baseline automatically	
Docker containerisation	One-command deployment across all plants and environments; consistent Python version	Medium

8 References

1. Brownlee, J., 2020. Deep Learning for Time Series Forecasting. Machine Learning Mastery.
2. Pedregosa, F. et al., 2011. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp.2825–2830.
3. Abadi, M. et al., 2016. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Available at: <https://www.tensorflow.org>
4. Box, G.E.P., Jenkins, G.M., Reinsel, G.C. and Ljung, G.M., 2015. Time Series Analysis: Forecasting and Control. 5th ed. Wiley.
5. Kaggle, 2020. Solar Power Generation Data. Available at: <https://www.kaggle.com/datasets/anikannal/solar-power-generation-data>
6. Hochreiter, S. and Schmidhuber, J., 1997. Long Short-Term Memory. Neural Computation, 9(8), pp.1735–1780.
7. Seabold, S. and Perktold, J., 2010. Statsmodels: Econometric and Statistical Modeling with Python. Proceedings of the 9th Python in Science Conference.
8. MNRE, 2023. Annual Report on Renewable Energy in India. Ministry of New and Renewable Energy, Government of India.
9. POSOCO, 2022. Report on Forecasting, Scheduling and Deviation Settlement Mechanism. Power System Operation Corporation of India.
10. Voyant, C. et al., 2017. Machine Learning Methods for Solar Radiation Forecasting: A Review. Renewable Energy, 105, pp.569–582.

9 Appendices

Appendix A — Consistency Checklist (Notebook / App / Report)

Item	Notebook v4	Web App	Report v7
Time-series model name	SARIMA (SARIMAX)	SARIMA	SARIMA throughout
Primary SARIMA granularity	Hourly (s=24, Sec 8)	Hourly (default toggle)	Sections 3.4.2, 6.3
Daily SARIMA alternative	Section 8b	Daily toggle option	Sections 3.4.2, 6.3 table
Daily residual analysis	Bar plot + histogram (2 panels)	Bar plot + histogram	Table in Section 3.4.2
Hourly residual diagnostics	plot_diagnostics() all 4 panels	plot_diagnostics() all 4 panels	Table in Section 3.4.2
EFFICIENCY excluded	Yes — commented in code	Yes — not in CANDIDATE list	Section 3.3 table, note
DAILY_YIELD excluded	Yes — commented in code	Yes — not in CANDIDATE list	Section 3.3 table, note
RF predict fix	rf.predict(X_test)	rf.predict(X_te)	Section 5.1 table
LSTM original epochs	50 epochs (original code)	N/A	Section 3.4.3

LSTM fixed epochs	Max 20 + EarlyStopping(patience=5)	N/A	Sections 3.4.3, 5.1 table
MAPE on daytime only	mask = y_true > 0	mask in compute_metrics()	Sections 4.3, 5.1 table

Appendix B — Notebook Structure (53 cells)

- Section 1: Environment Setup — imports, seeds, directories, dark plot theme
- Section 2: Plant Selection (PLANT_ID = 1 or 2 toggle) + Data Pipeline (load, merge, clean, feature engineering with EFFICIENCY and DAILY_YIELD excluded)
- Section 3: EDA — Spearman correlation heatmap, distributions, pairplot
- Section 4: Fault/Abnormality Detection — day-wise profiles, best/worst day comparison, CoV severity classification
- Section 5: Weather Analysis & Seasonal Decomposition (ambient temperature trend, seasonal decomposition, module temperature outlier boxplot)
- Section 6: Inverter Efficiency Analysis — from sensor DC_AC_RATIO and DELTA_TEMPERATURE (not from the excluded EFFICIENCY feature)
- Section 7: ML Models — LR, DT, RF with GridSearchCV + Gini importance + Permutation importance (10 repeats)
- Section 8: SARIMA hourly (s=24, ~816 pts, full four-panel residual diagnostics) + Section 8b: SARIMA daily (s=7, 34 pts, two-panel residual analysis only)
- Section 9: LSTM (max 20 epochs, EarlyStopping patience=5, Huber loss, sequence length=24)
- Section 10: All-Model Comparison + JSON Report | Section 10b: Plant 1 vs Plant 2 side-by-side comparison

Appendix C — Generated Output Files

- 01_correlation.png — Spearman correlation heatmap + feature correlation bars with AC_POWER
- 02_distributions.png — Feature distributions for daytime rows only (AC_POWER > 0)
- 03_pairplot.png — Bivariate relationships between AC_POWER, IRRADIATION, temperatures, DELTA_TEMPERATURE
- 04_daywise_dc_power.png — Per-day DC power profiles for all 34 days (fault pattern identification)
- 05_daily_totals.png — Daily AC power totals with best/worst day highlighted in green/red

- 06_best_vs_worst.png — Side-by-side DC power, irradiation, and temperature profiles for best vs worst day
- 07_weather_trends.png — 34-day trends for ambient temperature, module temperature, and irradiation
- 08_seasonal_decomp.png — Trend/seasonality/residual decomposition of ambient temperature (period=7 days)
- 09_module_temp_boxes.png — Module temperature box plots by day (outliers flag thermal fault events)
- 10_efficiency.png — DC_AC_RATIO vs output curve and DELTA_TEMPERATURE vs DC_POWER scatter (sensor-based, not EFFICIENCY feature)
- 11_feature_importance.png — Gini importance (left) and Permutation importance $\pm$ std (right) side-by-side
- 12_rf_evaluation.png — RF actual vs predicted scatter + residual plot
- 13_sarima_split.png — Hourly train/test split visualisation
- 14_sarima_forecast.png — SARIMA hourly forecast vs actual with 95% confidence interval band
- 15_sarima_diagnostics.png — Full four-panel residual diagnostics (hourly model only; all panels statistically valid at $n \approx 653$)
- 16_lstm_evaluation.png — LSTM training/validation loss curves, time-series overlay, scatter, error distribution
- 17_final_comparison.png — All ML models side-by-side MAE/RMSE/R² comparison bar charts
- metrics_report.json — Structured metrics with leakage notes, scale notes, best model selection, SARIMA configuration used

10 Auxiliaries

Please add the below-mentioned links.

Web link:

<https://solarpowerforecasting-7fmabm2xo5dkudkycnpdwu.streamlit.app/>

GitHub Repo:

https://github.com/chauhan22adarsh/Solar_Power_Forecasting

Data Source:

<https://www.kaggle.com/datasets/anikannal/solar-power-generation-data>

Required files: Plant_1_Generation_Data.csv, Plant_1_Weather_Sensor_Data.csv,
Plant_2_Generation_Data.csv, Plant_2_Weather_Sensor_Data.csv

Jupyter Notebook file:

https://github.com/chauhan22adarsh/Solar_Power_Forecasting/blob/main/Solar_Power_Forecasting.ipynb

Python file:

https://github.com/chauhan22adarsh/Solar_Power_Forecasting/blob/main/app.py